

NeuroBrain-Online

Physical Recovery Checkpoint & Disaster-Rebuild Manual

Purpose: rebuild the working NeuroBrain-Online robot even if the Raspberry Pi SD card, local files and all conversation history are lost. This document is intended to be stored inside the repository together with the source code.

Repository	drpeker/NeuroBrain-Online
Known online baseline	74802c4 - Initial NeuroBrain Online working multimodal system
Recovery documentation commit	328c4ea - complete engineering memory and disaster recovery
Frozen Python environment	f92ea14 - known-good Python package freeze
Target host	Raspberry Pi 4 Model B, 4 GB
OS baseline	Debian 13 (trixie), aarch64
Python baseline	Python 3.13.5
Primary runtime	/home/drpeker/neurobrain.py
Repository working copy	/home/drpeker/projects/NeuroBrain-Online
Virtual environment	/home/drpeker/oledtest

Do not confuse this repository with the older drpeker/NeuroBrain repository. They are different architectures and recovery instructions must not be mixed.

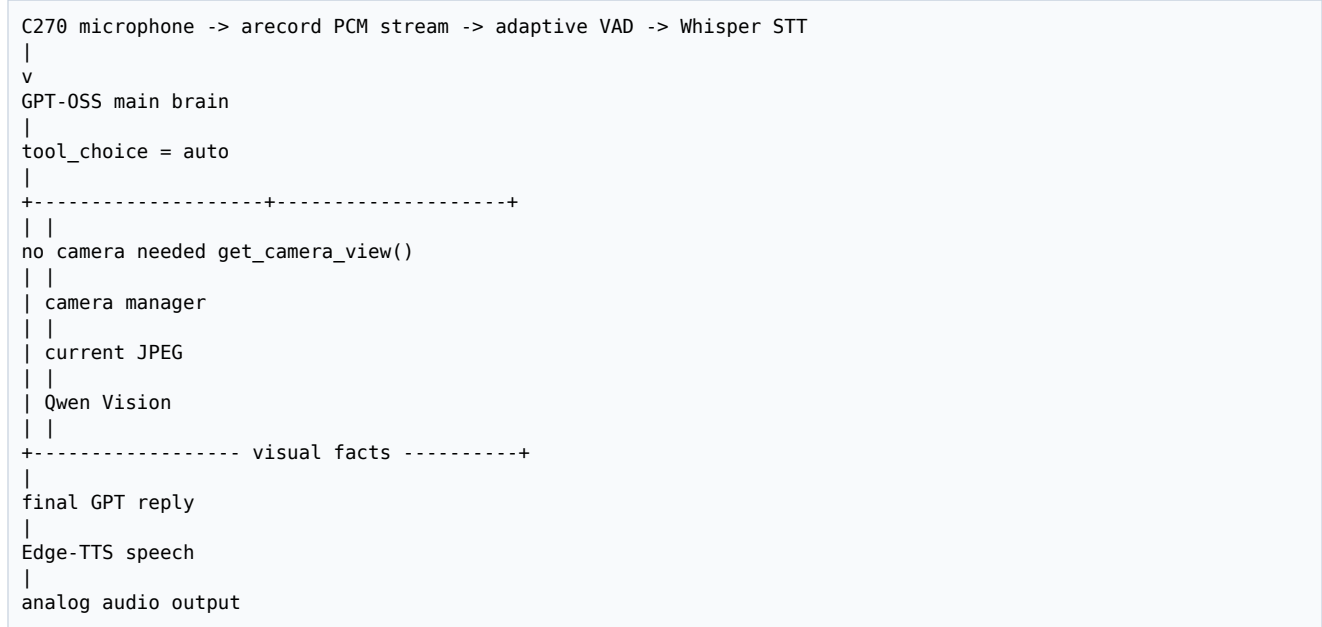
1. Recovery Philosophy

The repository is the engineering memory. A recovery must not depend on remembered chat messages, undocumented wiring guesses, local shell history, or files that existed only on the original SD card.

- Secrets are deliberately excluded. A valid GROQ_API_KEY must be recreated externally.
- Hardware identifiers such as ALSA card numbers may change after reinstall; identify devices again instead of blindly copying numbers.
- Do not invent motor GPIO pins. The motor API exists, but physical motor actuation remains intentionally inactive until pins and wiring are explicitly finalized.
- Preserve architecture before refactoring. A known-good checkpoint is more valuable than cosmetic cleanup.

2. Known-Good System Architecture

NeuroBrain-Online uses local Raspberry Pi hardware ownership with cloud cognition. Continuous audio/camera device ownership remains on the Pi. Groq services provide STT, reasoning and vision only when needed.



The camera is continuously open for local motion tracking, but a cloud vision request is made only when the main model decides current visual information is useful.

3. Physical Hardware Baseline

Computer	Raspberry Pi 4 Model B, 4 GB
Camera / microphone	Logitech C270; video normally /dev/video0; microphone originally ALSA hw:1,0
Audio output	Analog playback originally hw:0,0
OLED	SH1106, 128x64, I2C address 0x3C
Motor driver	MX1508 concept; motor functions currently stubbed and safe/inactive
Future controller	Seeed Studio XIAO ESP32-S3 Sense may be integrated later; it is not part of this baseline

Hardware verification commands

```
v4l2-ctl --list-devices
ls -l /dev/video*
arecord -l
aplay -l
i2cdetect -y 1
```

Expected OLED address is 0x3C. The C270 was /dev/video0 on the known system, but device numbering must be rechecked after OS reinstall.

4. Cloud Models and Exact Behavioral Settings

Provider	Groq / GroqCloud
STT model	whisper-large-v3-turbo
Main model	openai/gpt-oss-20b
Vision model	qwen/qwen3.8-27b
Main temperature	0.6
Main max_completion_tokens	200
Main reasoning_effort	low
Main include_reasoning	False
Vision temperature	0.2
Vision max_completion_tokens	300
Vision reasoning_effort	none
Turkish TTS	tr-TR-EmelNeural
English TTS	en-US-JennyNeural

Whisper automatically identifies Turkish or English. The reply should follow the user language. Obvious minor speech-recognition errors should be silently corrected from context. Do not inject the user name or title.

5. Voice Pipeline and VAD Checkpoint

MIC	hw:1,0 on the known host
RATE	16000 Hz
CHUNK_MS	20 ms
SAMPLES	320
BYTES	640
CALIBRATION_SEC	2.0
START_RATIO	3.0
START_MIN_ABOVE	180
START_HITS	5
END_RATIO	1.7
END_MIN_ABOVE	80
END_SILENCE_MS	300

MIN_SPEECH_MS	300
Prebuffer	15 chunks

Startup includes approximately one second of AGC warmup followed by two seconds of ambient-noise calibration. The noise floor adapts slowly during silence only.

Critical PCM read pattern

```
data = proc.stdout.read(BYTES - 0)
while len(data) < BYTES:
    more = proc.stdout.read(BYTES - len(data))
    if not more:
        break
    data += more
    if len(data) != BYTES:
        continue
```

This exact-chunk behavior is part of the known-good VAD path and should not be casually simplified.

6. TTS and Self-Trigger Protection

Edge-TTS binary	/home/drpeker/oledtest/bin/edge-tts
Temporary MP3	/tmp/neuro_tts.mp3
Temporary WAV	/tmp/neuro_tts.wav
Playback device	hw:0,0 on the known host
Post-playback acoustic tail	0.12 s

While Edge-TTS runs, while ffmpeg converts MP3 to WAV, and while aplay is speaking, the microphone PCM stream continues to be drained. This prevents the robot from hearing itself and avoids input backlog.

7. ALSA Overrun Protection

Long blocking cloud calls previously risked ALSA overrun. The known-good design runs the Groq call in a worker thread while the main thread continues consuming microphone PCM. Do not replace this with a simple blocking network call.

```
def groq_with_mic_drain(callable_func):
    result = {}
    error = {}
    def worker():
        try:
            result["value"] = callable_func()
        except Exception as e:
            error["value"] = e
    thread = threading.Thread(target=worker)
    thread.start()
    while thread.is_alive():
        data = proc.stdout.read(BYTES)
        if not data:
            break
    thread.join()
    if "value" in error:
        raise error["value"]
    return result["value"]
```

This wrapper is used around the initial GPT call, Qwen vision call and final GPT call after tool results.

8. Camera Manager Checkpoint

Owner	Single continuous cv2.VideoCapture(0) in neurobrain_modules/camera.py
-------	---

Requested resolution	320x240
Requested FPS	30
Capture buffer	1
Background subtraction	MOG2 history=100, varThreshold=30, detectShadows=False
Motion analysis size	160x120
Blur	GaussianBlur(5,5)
Contour area threshold	>120
Motion timeout	0.7 s before clear_motion()

The manager continuously stores the newest clean frame. `capture_image()` saves that existing latest frame and does not reopen the camera. This is critical: there should be one application-level camera owner.

9. Vision Tool Policy

The main GPT model decides autonomously whether to use `get_camera_view`. The user does not need to explicitly say “look” or “camera”. The model should use vision when current physical surroundings materially affect the answer, but must not use it for ordinary knowledge, coding, calculations or conversation.

- Never claim current physical sight unless the camera tool was actually called for the current request.
- Vision returns observations to the main GPT model; Qwen does not replace the conversational authority.
- The current design sends one current JPEG only. A future improvement may locally choose the sharpest of several frames, but that is not part of this checkpoint.

10. OLED Checkpoint

Driver	luma.oled SH1106
Address	0x3C
Display	128x64
Eye centers	(33,32) and (95,32)
Eye outline	approx. 25x23
Pupil radius	7
Motion clamp	x +/-16, y +/-12
Smoothing	0.75
Blink interval	3-6 s
Blink duration	~0.22 s

Public state API: `oled_start()`, `oled_idle()`, `oled_listening()`, `oled_thinking()`, `oled_speaking()`, `oled_error()`, `oled_stop()`, `set_motion()`, `clear_motion()`. Camera motion can drive gaze while the OLED thread remains independent.

11. Motor Safety Checkpoint

neurobrain_modules/motors.py intentionally exposes motor_left_forward(), motor_left_reverse(), motor_right_forward(), motor_right_reverse(), motors_forward(), motors_reverse() and motors_stop(), but the physical functions are currently stubs. Do not invent GPIO assignments during recovery.

12. Python Environment Snapshot

The known working virtual environment is /home/drpeker/oledtest using Python 3.13.5. The repository contains requirements.txt for core dependencies and requirements-lock.txt for the complete frozen environment.

groq	1.7.0
edge-tts	7.2.8
luma.core	2.6.0
luma.oled	3.16.0
opencv-python-headless	4.11.0.86
numpy	2.5.3
pillow	12.3.0
smbus2	0.6.1

For exact 2026 reproduction, prefer requirements-lock.txt. For migration to a newer OS/Python, core requirements may be installed first and compatibility tested deliberately.

13. Full SD-Card Disaster Recovery Procedure

Phase A - Rebuild host OS

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y git python3-venv python3-pip python3-opencv alsa-utils ffmpeg i2c-tools v4l-utils
```

Use a current 64-bit Raspberry Pi OS/Debian-compatible installation. Known baseline: Debian 13 trixie aarch64. Enable I2C and reboot if required.

Phase B - Verify hardware before application software

```
v4l2-ctl --list-devices
arecord -l
aplay -l
i2cdetect -y 1

arecord -D hw:1,0 -f S16_LE -r 16000 -c 1 -d 5 /tmp/mic.wav
aplay /tmp/mic.wav
```

If ALSA numbering differs, identify the C270 microphone and analog output again and update constants deliberately.

Phase C - Clone the correct repository

```
mkdir -p ~/projects
cd ~/projects
git clone https://github.com/drpeker/NeuroBrain-Online.git
cd NeuroBrain-Online
git log --oneline --decorate -10
```

Phase D - Rebuild Python environment

```
python3 -m venv --system-site-packages ~/oledtest
source ~/oledtest/bin/activate
python -m pip install --upgrade pip
python -m pip install -r ~/projects/NeuroBrain-Online/requirements-lock.txt
```

If the frozen lock cannot be installed on a future Python version, recreate the original Python 3.13-compatible environment or fall back to requirements.txt and test all acceptance criteria.

Phase E - Restore secret outside Git

```
export GROQ_API_KEY="YOUR_GROQ_API_KEY"
```

Never commit the key to Git, documentation, shell scripts, or Python source.

Phase F - Restore live runtime

```
cp ~/projects/NeuroBrain-Online/neurobrain.py ~/neurobrain.py
rm -rf ~/neurobrain_modules
cp -a ~/projects/NeuroBrain-Online/neurobrain_modules ~/neurobrain_modules
```

Phase G - Compile before run

```
python -m py_compile \
~/neurobrain.py \
~/neurobrain_modules/camera.py \
~/neurobrain_modules/oled.py \
~/neurobrain_modules/motors.py
```

Phase H - Run

```
source ~/oledtest/bin/activate
export GROQ_API_KEY="YOUR_GROQ_API_KEY"
python ~/neurobrain.py
```

14. Acceptance Test - A Recovery Is Not Complete Until All Pass

- OLED eyes animate normally.
- Camera remains continuously available and local motion moves gaze.
- VAD remains quiet in normal room noise and starts reliably on speech.
- Turkish speech transcribes and receives a Turkish reply.
- English speech transcribes and receives an English reply.
- Edge-TTS uses Emel for Turkish and Jenny for English.
- The robot does not immediately hear its own TTS as a new command.
- Ordinary knowledge questions do not invoke camera unnecessarily.
- Implicit visual questions can cause GPT to call get_camera_view.
- The robot never claims current sight without a current camera call.
- Long vision/reasoning calls do not produce ALSA overrun.
- Ctrl+C shuts down camera, OLED, arecord and motor API cleanly.
- Motor functions remain physically inactive until explicitly implemented.

15. Known Local Backups From the Original Host

These paths existed on the original SD card and are historical checkpoints. They are useful only if that original filesystem still exists; disaster recovery must not depend on them.

- /home/drpeker/neurobrain_BREAKPOINT_VOICE.py (+ .sha256)
- /home/drpeker/neurobrain_BREAKPOINT_VISION_TOOL.py (+ .sha256)
- /home/drpeker/neurobrain_BREAKPOINT_VISION_NO_OVERRUN.py (+ .sha256)
- Historical before_* backups for Edge-TTS, Groq, fast loop, multimodal, OLED/camera and vision integration.

16. Local/Offline Model Archive - Not Used by Current Online Architecture

The older/offline model collection may exist under /home/drpeker/neurobrain/models. These models are not required by the current Groq-backed online system and should not be confused with the active cloud models.

- qwen3-1.7b-q4km.gguf
- qwen2.5-1.5b-instruct-q4km.gguf
- qwen3-0.6b-q4.gguf
- qwen3-0.6b-tools-q4km.gguf
- lfm2.5-1.2b-instruct-qad-q4.gguf
- Phi-4-mini-reasoning-Q4_K_M.gguf

17. What Must Never Be Reconstructed From Guesswork

- motor GPIO pin numbers
- new OLED wiring if the physical wiring is unknown
- alternate model names
- different TTS voices without testing
- keyword-based camera routing as a substitute for tool selection
- hard-coded API credentials
- a second process opening the camera concurrently

18. Repository Files That Form the Recovery Set

- neurobrain.py
- neurobrain_modules/camera.py
- neurobrain_modules/oled.py
- neurobrain_modules/motors.py
- requirements.txt
- requirements-lock.txt
- README.md
- docs/ARCHITECTURE.md
- docs/AUDIO_PIPELINE.md
- docs/CAMERA_VISION.md
- docs/CURRENT_STATE.md
- docs/DESIGN_DECISIONS.md
- docs/HARDWARE.md
- docs/INSTALL.md
- docs/MODELS.md
- docs/RECOVERY.md

- docs/SD_CARD_DISASTER_RECOVERY.md
- docs/SOFTWARE_INVENTORY.md
- docs/SYSTEM_DIAGRAMS.md
- install_neurobrain_online_project_memory.sh
- this PDF

19. Final Recovery Rule

After every physically verified architectural change, update the source code and the recovery documents in the same development cycle. The system is not considered safely evolved until the new working state is recoverable without relying on memory or chat history.